


# Introducing Hyphenate.

The autonomous finance platform.

trusted-by GROUPON INSTRUCTURE SIERRA Quantum eightfold.ai minimelts Kiteworks cadence HiBid DURA SOFTWARE & more

backed-by foundation capital gaia VENTURES BOLD CAP Microsoft OpenAI MongoDB. SAP KPMG EY perplexity ZUORA

THE NEXT 60 MINUTES

# From a capable agent to a trusted autonomous system.

| | |
|---|---|
| 0:01 | Hyphenate, and our platform video |
| 0:07 | The track, prizes and judging |
| 0:15 | Systems of record vs. a system of work |
| 0:23 | SaaS vs. outcome-based pricing |
| 0:26 | Building a good system of work |
| 0:45 | Strong vs. weak submissions |
| 0:47 | Your weekend with Hyphenate |
| 0:50 | Q&A |

Questions welcome in Zoom chat any time.


Autonomous Finance

# The Hyphenated Enterprise: autonomous systems for finance.

Build the system a finance team would bring on board. No finance background needed.

01

## Pick a company.

Any company: a sports team, a coffee chain, a Fortune 500.

02

## Pick a few roles.

Billing, Payroll, Revenue, the Forecast. Your call.

03

## Make it autonomous.

AI that does the work on its own and explains every call.

# What you win, and how to win it.

1st-place

**$8,000**

Cash prize, per team

2nd-place

**$5,000**

Cash prize, per team

3rd-place

**$3,000**

Cash prize, per team

Top 5 teams: an expedited interview at Hyphenate for every member.

how-we-judge

01 Is it real enough?

02 Would you hire it?

03 Did we see something new?

04 Did it have a “wow” moment?



scan-for-the-full-brief

office-hours

**Fri, 7:00 PM**

Theater · technical workshop & office hours

THE OFFICE OF THE CFO

# What a finance team actually does.

| | | | | |
|---|---|---|---|---|
| **Revenue**<br>Contracts become revenue schedules | **Receivables**<br>Payments in, short or late; who to chase | **Cash**<br>Every bank line matched to what it pays | **Payables**<br>Bills, stalled approvals, bank changes | **Accruals**<br>Estimate, confirm, reverse |
| **Expenses**<br>Prepaids, assets and leases over time | **Payroll**<br>Runs and grants into entries | **Close**<br>Reconciliations, controls, sign-offs | **FP&A**<br>Budgets, forecasts, variances | **Reporting**<br>Statements, metrics, board packs |

Ten of many. Pick a corner of one: one corner done deep beats ten done shallow.

# One refund touches five systems.

ONE ECONOMIC EVENT

**A customer gets a refund.**

| | |
|---|---|
| Billing | A credit note; the invoice reopens |
| Bank | Cash leaves; a payout to match |
| General ledger | Revenue reverses; receivables move |
| CRM | The customer's balance changes |
| Forecast | Next quarter's numbers shift |

Every number has to agree with every other one, and nothing moves without the right sign-off.

BEYOND AGENTIC HARNESSES

# Harnesses made agents capable.

Model + Tools + Loop + Context

Agent SDKs, MCP, coding agents: an agent that can do the task.

Now put it inside a company. Four questions show up on day one:

01 · WORLD
Where does it act?

02 · TIME
What does it remember next quarter?

03 · TEAM
Who does it work with?

04 · LIMITS
What may it do, and can it prove why?

THE ONE IDEA

# Records hold the truth. A system of work holds the work.

SYSTEMS OF RECORD · VERTICAL DATABASES

## The truth about the business.

Bank · ERP and ledger · contracts · CRM · inbox

Owned by finance and IT. They disagree with each other. You integrate with them; you don't replace them.



**Gateway**

One door. Every write checked.

SYSTEM OF WORK · EXECUTION PLATFORM

## The truth about the work.

Work items · decisions and reasons · questions and approvals · learned policies · the log

Owned by you. Agents and people share it. This is what you build.



GitHub holds the record.

IN CODE



The work runs here.

WHY NOW · HOW SOFTWARE GETS PAID

# SaaS pricing sells access. Outcome-based pricing sells work.

SAAS · PER USER

Oracle NetSuite

## ~$125–199

per full user a month, plus a base fee. Oracle doesn't publish prices; these are industry estimates.

OUTCOME-BASED · PER RESULT

Intercom · Zendesk

## $0.99

per conversation Fin resolves. Zendesk: $1.50–2.00 per automated resolution. No result, no bill.

HYPHENATE · ON THE WORK

Hyphenate

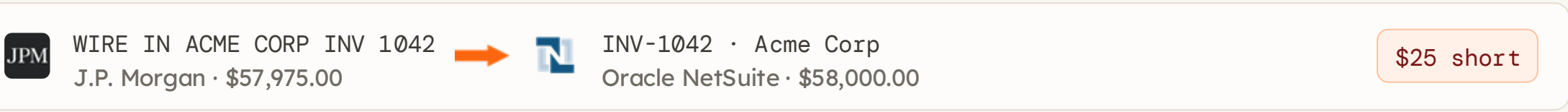
## Not per login

Priced per function or workstream: by the work it takes off the team, not who signs in.

Billing per outcome means proving every outcome. That takes a system of work.

THE BLUEPRINT · ONE WIRE, END TO END

# A good system of work learns, improves and proves.




**01 · SELF-LEARNING**
Learns the job at onboarding

**02 · SELF-IMPROVING**
Gets better with every correction

**03 · AUDIT TRAIL**
Proves every step, months later

**ENVIRONMENT**
Sync the records, statefully

**AGENTS + HUMANS**
Map actors, actions, resources

**THE HARNESS**
Tools are CRUD on the work

**TRACEABILITY**
Log every transition as a diff

01 · SELF-LEARNING, AT ONBOARDING

# Learn the job from the company's history.

HUMAN + SYSTEM OF RECORD

A new accountant shadows for two weeks, reads last year's close binders and asks: what do we do with short wires? The answer lives in her head.

AI AGENT + SYSTEM OF WORK

Queries a year of Oracle NetSuite payments against J.P. Morgan lines, finds how short wires were booked, and drafts a policy.

`pol-001 · wire ≤ $50 short → 6850 Bank Fees`

```
▸ ai_agent  think
  How did this team book wires that came in short?
▸ ai_agent  run_sql
  SELECT account, COUNT(*) AS n, MAX(gap) AS max_gap
  FROM   paid_invoices_vs_bank            -- 12 months
  WHERE  memo LIKE 'WIRE IN %' AND gap <= 50
  GROUP  BY account;
◂ 6850 Bank Fees | 214 | 48.00
▸ ai_agent  propose_policy
  pol-001: wire ≤ $50 short → book gap to 6850
  evidence: 214 precedents · approver: human_controller
◂ needs_approval
```

02 · SELF-IMPROVING, EVERY DAY

# Every correction becomes precedent.

HUMAN + SYSTEM OF RECORD

Acme starts paying two invoices with one wire. She splits it by hand and remembers. When she leaves, the lesson leaves with her.

AI AGENT + SYSTEM OF WORK

An accountant corrects its match. The agent finds similar corrections, spots the pattern, and proposes a general policy.

pol-014 · one wire, many invoices → split

```
◂ human_accountant corrected WI-5602
  split one wire across INV-1077 and INV-1078
▸ ai_agent  search_precedents(customer="Acme")
◂ 3 alike: several invoice numbers in one memo
▸ ai_agent  think
  Memo lists the invoices; amounts add up to
  the wire. Not Acme-specific: make it general.
▸ ai_agent  propose_policy
  pol-014: split a wire across every invoice
  number in its memo when the amounts add up
  evidence: prec-311, prec-318, prec-322
◂ needs_approval · human_controller
```

03 · AUDIT TRAIL, FROM DAY ONE

# Months later, “why?” is one query.

HUMAN + SYSTEM OF RECORD

February audit: why was INV-1042 closed $25 short? Oracle NetSuite shows a payment and a write-off. The why is in someone’s inbox.

AI AGENT + SYSTEM OF WORK

Every step was an event: the bank line, the invoice, the policy and its version, who approved, what changed. The answer is a query.

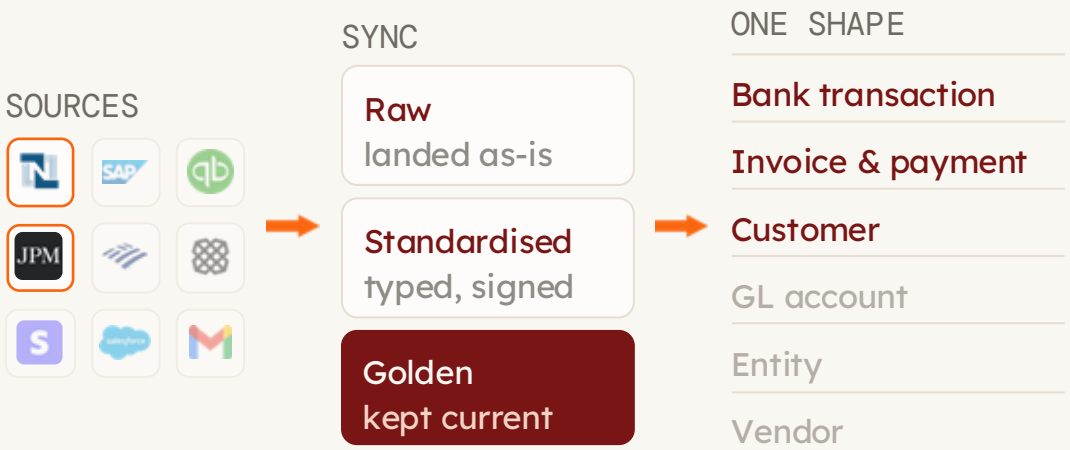
append-only · written with the change

```
SELECT at, actor, change, why
FROM   audit_log
WHERE  subject = 'INV-1042'
ORDER BY at;

at     actor              change      why
09:02  sync               + NEW       JPM-88310442
09:03  ai_agent           → PROPOSED  pol-001 v1, $25 gap
11:40  human_accountant   → APPROVED  "intermediary fee"
11:41  gateway            → POSTED    NetSuite PYMT-7781
11:41  gateway            + JE        $25.00 → 6850
```

BUILD 1 · THE ENVIRONMENT

# Sync the systems of record into one shape.




STATEFUL, NOT A ONE-OFF EXPORT

Change-only sync with a cursor per source. Stable ids, so a re-sync never duplicates. Late bank posts re-read. Every golden row traces to its raw row.

```
CREATE TABLE bank_transaction (
  id         TEXT PRIMARY KEY, -- md5(acct|src_id)
  source     TEXT,             -- 'jpm'
  txn_date   DATE,
  amount     NUMERIC(18,2),     -- 57975.00, in > 0
  memo       TEXT,             -- 'WIRE IN ACME…'
  raw_ref    TEXT              -- source row
);
CREATE TABLE invoice (
  id         TEXT PRIMARY KEY, -- 'INV-1042'
  customer   TEXT,             -- 'Acme Corp'
  amount     NUMERIC(18,2),     -- 58000.00
  status     TEXT,             -- 'Open'
  raw_ref    TEXT              -- source row
);
CREATE TABLE sync_cursor (source, cursor, at);
```

# Map who may do what, to which record.

| ACTORS | ACTIONS | RESOURCES |
|---|---|---|
| Sync job | Create | Bank line |
| AI agent | Propose | Invoice |
| Accountant | Ask · answer | Work item |
| Controller | Approve | Policy |
| Gateway | Post | Journal |

OUR WIRE · CHECKED ON EVERY CALL

| | | | |
|---|---|---|---|
| sync | create | WI-5531 | ✓ |
| ai_agent | propose | WI-5531 | ✓ |
| ai_agent | approve | WI-5531 | ✗ |
| human_accountant | approve | WI-5531 | ✓ |
| ai_agent | post | NetSuite | ✗ |

```
ROLES = {  # role -> resource -> allowed actions
  "sync":             {"work_item": {"create"}},
  "ai_agent":         {"work_item": {"propose", "ask"},
                       "policy":    {"propose"}},
  "human_accountant": {"work_item": {"approve", "answer"}},
  "human_controller": {"policy":    {"approve"}},
}  # no role can "post": only the gateway does

def can(actor, action, res) -> bool:
    if action not in ROLES[actor.role].get(res.kind, ()):
        return False
    if action == "approve" and res.proposed_by == actor.id:
        return False                # separation of duties
    return True
```

# The agent's tools are CRUD on the work.

NEW — sync: a new bank line

↓ propose · ai_agent

PROPOSED — ask ⇄ answer — NEEDS_INFO

↓ approve · human_accountant

APPROVED

↓ post · gateway → NetSuite

POSTED — the only write to the record

| | |
|---|---|
| C | sync creates a work item per bank line |
| R | tools read records, policies, precedents |
| U | every change goes through transition() |
| D | never: a reversal is a new transition |

```
TRANSITIONS = {  # (state, action) -> next state
  ("NEW",         "propose"): "PROPOSED",
  ("PROPOSED",    "ask"):     "NEEDS_INFO",
  ("NEEDS_INFO", "answer"):  "PROPOSED",
  ("PROPOSED",    "approve"): "APPROVED",
  ("APPROVED",    "post"):    "POSTED",    # NetSuite
}

@tool  # the harness sees tools, not tables or keys
def propose_match(item_id, invoice_id, policy_id):
    item = work_items.read(item_id)                  # R
    return transition(item, "propose", AI_AGENT,     # U
        patch={"invoice": invoice_id,
               "policy": policy_id},
        reason=f"{policy_id}: gap within tolerance")
```

# Log every transition as a diff.

```
11:40 · human_accountant · approve
subject   WI-5531 · INV-1042
diff      state: PROPOSED → APPROVED
          approved_by: ∅ → human_accountant
reason    "intermediary fee"
evidence  JPM-88310442
          INV-1042 · pol-001 v1
```

Append-only: fixes are new events.

Same commit as the change itself.

Denied attempts are logged too.

```python
def transition(item, action, actor, patch=None, reason=""):
    to = TRANSITIONS.get((item.state, action))
    ok = to is not None and can(actor, action, item)
    before = snapshot(item)
    if ok:
        item.update(patch or {}, state=to)
    with db.transaction():              # one commit
        if ok: db.save(item)
        db.append("audit_log", dict(    # append-only
            at=now(), actor=actor.id, action=action,
            subject=item.id, allowed=ok, reason=reason,
            diff=diff(before, snapshot(item)),
            evidence=item.evidence))    # source row ids
    if not ok: raise Denied(action, item.id)
    return item
```

WHAT WE LOOK FOR

# Strong vs. weak submissions.

STRONG

A real company with a messy corner, and an answer key

A few roles done deep, named in one line

Asks at the right moment, and handles a no

Limits enforced in code; a record of every call

A moment on stage, and a score to back it

WEAK

A chatbot over a spreadsheet, or a one-shot extractor

Ten workflows, none of them deep

Clean happy-path data that only works on the script

An admin key, and “the prompt tells it not to”

Memory that ends with the context window; “trust me”

# Your weekend with Hyphenate.

FRI · 7:00 PM

Technical workshop & office hours with our founding engineers

Theater

SAT · 12–2 PM

Invite-only lunch for 2027 grads

Drop your resume at the booth

SAT · 2–4 PM

Boba and cookies mixer, open to all

Mixer Area, Gate 6

ALL WEEKEND

Our booth: questions, jobs, hoodies

While they last

WE'RE HIRING

## Software Engineer, New Grad

Full-time · starts 2027 · New York

scan-to-apply

WE'RE HIRING

## Software Engineering Intern

Full-time · Summer 2027 or earlier · New York

scan-to-apply

HYPHENATE

# Questions?

Build the system a finance team would bring on board.

IN-PERSON WORKSHOP

Fri, Oct 23 · 7 PM · Theater

BOOTH

All weekend · jobs, hoodies, help

SLACK

#cohost-hyphenate